ClonoForge enables deterministic and efficient UMI consensus generation for immune repertoire sequencing

Yoon Sing Yap

Abstract

Unique molecular identifiers (UMIs) are widely used in immune repertoire sequencing to correct PCR amplification errors and sequencing artifacts. Generating consensus sequences from UMI read groups requires careful treatment of base calls, quality scores, reproducibility, and resource limits. Here, we present ClonoForge v1.0, a focused FASTQ preprocessing and quality-aware consensus engine implemented in Rust. Evaluated on prepared sequencing inputs from a Myasthenia gravis B-cell repertoire study (SRR4026043) and an Influenza-vaccination B-cell repertoire study (SRR4431771), ClonoForge reproduces the complete pRESTO 0.7.9 consensus FASTQ outputs byte for byte, ensuring exact preservation of base calls, quality scores, headers, and record order. On a tested Apple ARM64 workstation, ClonoForge demonstrated lower median elapsed runtime across three measured runs from separate campaigns: 64.984 s versus 115.915 s for the Myasthenia gravis study (1.78× faster), and 53.259 s versus 126.500 s for the Influenza-vaccination study (2.38× faster). ClonoForge enforces a default standard limit of 50 reads per group, with opt-in experimental profiles supporting up to 500 reads (tested on authentic reads up to 497 and synthetically at 500 reads). As a focused consensus component rather than a full pRESTO replacement, ClonoForge reduced observed consensus runtime on the tested inputs with byte-level concordance, without asserting new biological findings.

Introduction

Adaptive immune receptor repertoire sequencing (AIRR-seq) profiles the diversity of antibody and T-cell receptor repertoires to elucidate immune responses in vaccination, infection, and autoimmunity. To overcome technical noise arising from PCR amplification bias and sequencing errors, modern protocols tag individual template molecules with unique molecular identifiers (UMIs). Grouping reads by molecular identifier and collapsing them into consensus sequences enables error-reduced reconstruction of sequenced receptor templates.

Consensus generation from molecular groups introduces three critical computational challenges. First, downstream clonal clustering, germline gene annotation, and somatic hypermutation analyses are sensitive to single nucleotide calls and Phred quality scores; software comparisons therefore need to check both base calls and quality scores. Second, read-group sizes can vary substantially within immune-repertoire libraries, making explicit limits important for predictable execution. Third, reproducible bioinformatics demands strict determinism so that identical inputs consistently produce identical byte streams across runs. While comprehensive software suites such as pRESTO [1] provide versatile workflows for lymphocyte receptor analysis, dedicated consensus implementations can mitigate performance bottlenecks while maintaining strict algorithmic compatibility.

We developed ClonoForge, a focused, bounded, and deterministic FASTQ preprocessing and consensus workflow implemented in Rust. ClonoForge combines bounded disk-backed external sorting with pRESTO's quality-aware consensus scoring model. Here, we evaluate ClonoForge against pRESTO 0.7.9 using prepared inputs from the Myasthenia gravis B-cell repertoire study (SRR4026043) and the Influenza-vaccination B-cell repertoire study (SRR4431771). We show that ClonoForge achieves byte-identical consensus FASTQ outputs with reduced runtime on an Apple ARM64 architecture, supporting its use as a focused consensus implementation for the tested inputs and settings.

Results

ClonoForge workflow and architectural design

ClonoForge implements a preprocessing and consensus pipeline tailored for paired-end immune repertoire sequencing libraries (Figure 1a). The workflow ingests paired FASTQ records, validates read pairing, extracts exact molecular identifiers, trims target sequences, groups eligible reads sharing identical barcodes, and computes quality-aware consensus sequences. Configurable parameters enable users to specify barcode locations, mate orientations, and optional anchored primer matching or ungapped overlap assembly for specific library geometries. Barcode extraction operates on exact string matches without heuristic error correction.

Three core architectural principles guide ClonoForge's implementation: scientific compatibility with reference consensus logic, bounded and deterministic execution, and computational performance (Figure 1b). Bounded external sorting and explicit admission limits reduce dependence on whole-dataset in-memory processing and make resource use more predictable under the configured limits. Deterministic tie-breaking and ordered stream execution support repeatable output bytes under the tested settings. Each run generates consensus FASTQ records accompanied by cryptographic integrity metadata and structured execution summaries. The implementation uses Rust data structures and bounded processing around the defined consensus calculation.

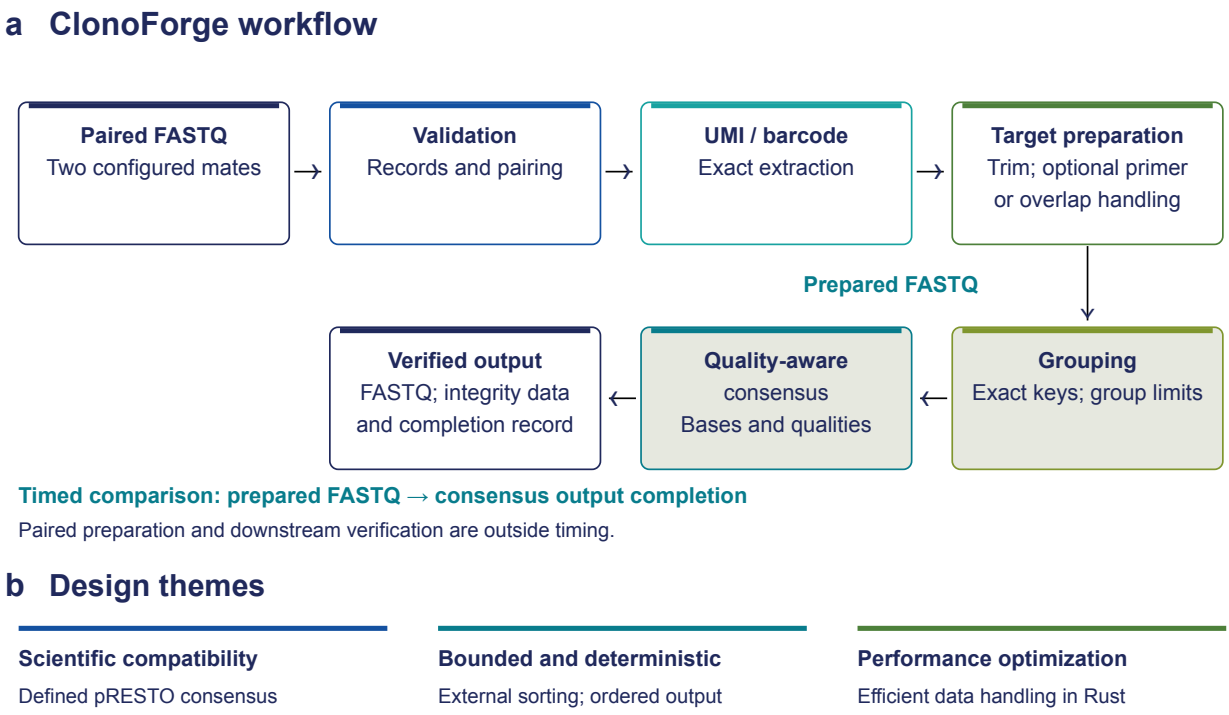


Figure 1. ClonoForge workflow and design themes. (a) The paired FASTQ workflow progresses through record validation, exact UMI/barcode extraction, target preparation, read grouping, quality-aware consensus generation, and verified output. Optional target preparation accommodates user-specified assay geometries such as anchored primers or overlap assembly. (b) Three core design themes govern implementation: scientific compatibility with defined reference consensus logic, bounded and deterministic execution via external sorting, and computational performance optimization in Rust. Shading highlights the grouping and consensus stages. The timed benchmark comparison spans from prepared FASTQ input through consensus output completion; upstream paired-read preparation and downstream verification are excluded from timing. The default standard limit is 50 reads per group; larger limits are available as experimental profiles.

Exact byte-level consensus concordance

To verify that ClonoForge preserves reference consensus behavior, we compared its outputs against pRESTO 0.7.9 using matched prepared inputs from the Myasthenia gravis study (SRR4026043) and the Influenza-vaccination study (SRR4431771) (Table 1; Figure 2a). Across 920,842 prepared reads in the Myasthenia gravis study (yielding 162,693 consensus records) and 539,013 prepared reads in the Influenza-vaccination study (yielding 452,381 consensus records), ClonoForge produced complete consensus FASTQ outputs that were byte-identical to pRESTO across all validated warmup and measured runs.

Record-level mean Phred qualities agreed for all compared records (Spearman $\rho = 1.000000$ in each study; Figure 2a). Correlation alone does not establish identity. Exact byte-level concordance shows that, for the tested inputs and settings, replacing pRESTO consensus generation with ClonoForge would not change the consensus FASTQ supplied to downstream analyses.

Table 1. Matched prepared inputs and consensus outputs. Counts reflect the prepared computational boundary rather than total raw sequencing reads. Full input and output SHA-256 digests are provided in the accompanying concordance records.

Study / accession	Prepared reads	Consensus records	Output agreement
Myasthenia gravis study (SRR4026043)	920,842	162,693	Byte-identical
Influenza-vaccination study (SRR4431771)	539,013	452,381	Byte-identical

Runtime performance at the consensus boundary

At the prepared-FASTQ-to-consensus boundary, ClonoForge demonstrated lower median elapsed runtime across both test datasets on a tested Apple ARM64 workstation configured with a single worker thread (Table 2; Figure 2b). For the Myasthenia gravis study, ClonoForge achieved a median runtime of 64.984 s (observed range: 64.881–65.247 s) compared to 115.915 s for pRESTO (observed range: 115.646–116.764 s), yielding a 1.78× speed ratio. For the Influenza-vaccination study, ClonoForge achieved a median runtime of 53.259 s (observed range: 52.737–53.290 s) compared to 126.500 s for pRESTO (observed range: 123.739–130.438 s), yielding a 2.38× speed ratio.

These timings were measured across three sequential runs per tool from separate campaigns on the same host, inputs, one-worker configuration, timed boundary, and validation rules. Observed minimum-to-maximum ranges reflect empirical variation; because host caching, thermal conditions, and background activity were uncontrolled, these descriptive ranges represent observed performance rather than statistical confidence intervals.

Table 2. Runtime at the prepared-FASTQ-to-consensus boundary. Elapsed times are in seconds across three measured sequential runs per tool. pRESTO was evaluated on version 0.7.9.

Study (accession) / tool	Run 1	Run 2	Run 3	Median	Min–max
Myasthenia gravis (SRR4026043) / ClonoForge	64.984	64.881	65.247	64.984	64.881–65.247
Myasthenia gravis (SRR4026043) / pRESTO	115.915	115.646	116.764	115.915	115.646–116.764
Influenza vaccination (SRR4431771) / ClonoForge	53.290	53.259	52.737	53.259	52.737–53.290
Influenza vaccination (SRR4431771) / pRESTO	126.500	123.739	130.438	126.500	123.739–130.438

Opt-in experimental profiles for larger read groups

The standard v1.0 limit is 50 reads per group. Limits of 100, 250, and 500 are opt-in experimental profiles. In authentic reads from a separate B-cell repertoire study (SRR1383459), the largest admitted groups contained 100, 250, and 497 reads at the respective experimental limits. Exactly 500 reads was tested synthetically. Qualified consensus-group counts were 31,440, 31,868, 32,306, and 32,575 at limits of 50, 100, 250, and 500, respectively (supporting group-size qualification data). Neither prepared runtime input admitted a group above 50 reads. These finite evaluations found concordant consensus generation and filtering for the tested deeper groups.

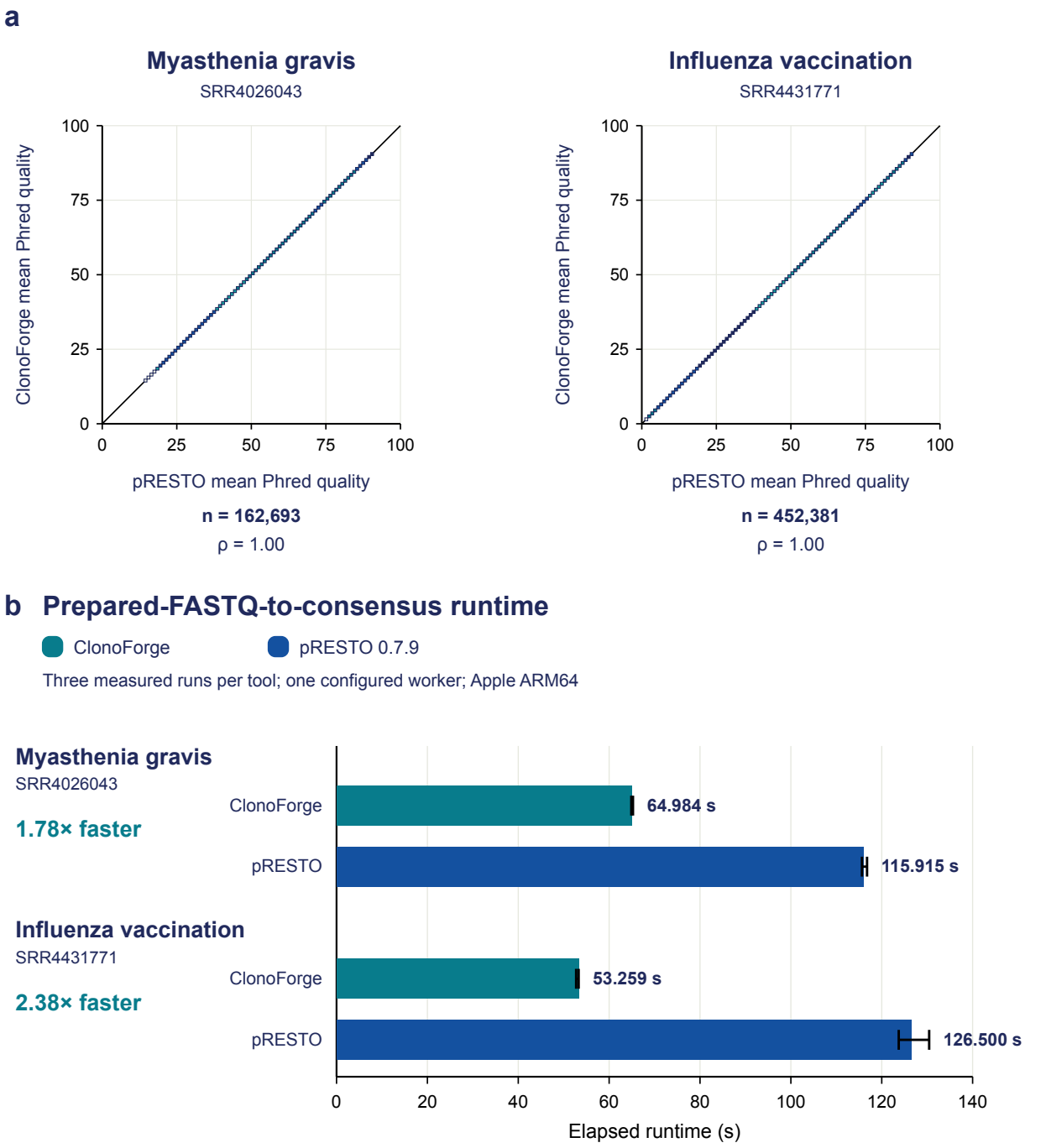


Figure 2. Output concordance and runtime comparison. (a) Record-level comparison of mean consensus Phred quality for the myasthenia gravis and influenza-vaccination studies. The diagonal line indicates identity. Spearman ρ and the number of compared records are shown in each plot. Complete consensus FASTQ outputs were byte-identical; the correlation plots provide a visual summary of record-level agreement. (b) Median elapsed runtime across three measured runs. Black error bars show the observed minimum-to-maximum range, not confidence intervals. Individual measurements are reported in Table 2. Timings came from separate campaigns and were not paired trials.

Discussion

ClonoForge reduced observed UMI consensus runtime while matching pRESTO 0.7.9 consensus FASTQ bytes across the tested inputs from the Myasthenia gravis study and the Influenza-vaccination study. This agreement checks base calls, quality scores, headers, and record order at the tested consensus boundary. It does not establish equivalence across untested inputs or complete analytical workflows.

The observed runtime differences—1.78× for the Myasthenia gravis study and 2.38× for the Influenza-vaccination study on Apple ARM64—reflect the performance of the complete implementations at the tested boundary. The present study did not isolate the contribution of external sorting, implementation language, data structures, or workflow orchestration. Bounded external sorting and explicit admission limits reduce dependence on whole-dataset in-memory processing and make resource use more predictable under the configured limits.

These findings reflect defined computational boundaries. ClonoForge is designed specifically as a focused prepared-FASTQ-to-consensus engine rather than a comprehensive replacement for the broader pRESTO toolkit. Upstream preparation (such as SRA extraction, base quality filtering, and primer trimming) and downstream analyses (including V(D)J annotation and clonal clustering) remain essential functions provided by broader toolkits like Immcantation. Furthermore, the evaluated datasets serve strictly as computational benchmarks without reinterpreting source-study biological findings, and observed runtime ratios on Apple ARM64 may vary across different hardware environments. Because preparation excluded many reads from the influenza-vaccination input under the exact-key, group-size, and equal-length rules, this computational subset must not be interpreted as a measure of biological recovery.

The standard profile limits groups to 50 reads. Opt-in experimental profiles allow limits of 100, 250, and 500 reads, with evidence limited to the tested cases. Together, these results establish ClonoForge as a focused prepared-FASTQ-to-consensus implementation that reproduced pRESTO output bytes and reduced observed runtime on two tested inputs using the evaluated Apple ARM64 system.

Methods

Consensus and processing

Reads are grouped by an exact barcode. The paired preparation excludes whole groups that exceed the selected limit or contain unequal target lengths. Standard consensus accepts uppercase A/C/G/T/N and Phred qualities from 0 to 93.

For a position with multiple reads, let S(b) be the sum of Phred qualities for observations of base b, where b is A, C, G, or T. Let T be the sum of qualities of all observations at that position, including N. The score is

$$\text{score}(b) = \text{floor}(S(b)^2 / T) \text{ for } T > 0; \text{ otherwise score}(b) = 0.$$

Thus, the sum for each base is squared before division; the calculation does not sum squared individual qualities. Integer division takes the floor. Among observed A/C/G/T bases, the base with the largest score wins; ties follow A/C/G/T order. If there is no observed A/C/G/T base, the result is N with quality zero. The winning quality is the smaller of its score and 90.

The default frequency threshold is 0.6, using the winning base's count divided by the number of non-N observations. Minimum quality is zero and minimum group count is one. A position that fails frequency or minimum quality becomes N with quality zero. Singleton handling preserves its input quality. These rules describe the implemented reference-compatible operation, not a posterior probability of molecular correctness.

Processing and resource limits

ClonoForge uses external sorting, configured record and resource limits, and ordered execution. Resource admission is a model, not a strict operating-system memory guarantee. Completed output carries integrity metadata and a completion record. Independent numerical cases, deterministic-output tests, and failure tests supplement the authentic-read comparisons. Optional overlap assembly and primer handling were outside the matched consensus benchmark.

Public inputs and preparation

For the Myasthenia gravis B-cell repertoire study (SRR4026043; PRJNA338795; SRX2018082), the sequencing run contains 1,332,467 paired spots [2]. Full-quality SRA extraction was used. Its first 25,000 pairs matched the official example archive byte for byte [4]. The molecular identifier occupies the first 17 bases of read 2. pRESTO quality, primer, and pairing preparation yielded 987,171 paired records. Exact-key preparation then excluded 66,329 reads in 5,138 unequal-length groups, leaving 920,842 reads in 162,693 groups. No group exceeded 50.

For the Influenza-vaccination B-cell repertoire study (SRR4431771; PRJNA349143; SRX2251665), the sequencing run contains 1,853,737 paired spots [3]. Preparation used the first 17 bases of read 2 as the exact key and read 1 as the target, without primer or quality filtering. It excluded 2,421 invalid-key reads and 1,312,303 reads in oversized or unequal-length groups. The retained input contained 539,013 reads in 452,381 groups. The raw size-131 group failed the equal-length requirement even at a larger limit.

Both prepared inputs were independently constructed and checked by SHA-256 before comparison.

Software identity, timing, and validation

The evaluated ClonoForge v1.0 binary was built from commit `c20a8a127d2fdccd35b9af7d8c65c6d806d2ada0`. Its SHA-256 was `1ee1a2033ebc659f14f2713f8fbff3b4e8c5adf2621f9bbd8a97d3d4f8bbc01c`. The reference was pRESTO 0.7.9 on CPython 3.9. The host used Apple ARM64 and macOS 27.0.1. Each tool used one configured worker. Numerical-library thread settings were fixed to one.

Each tool had one warmup and three sequential measured runs per accession. The final ClonoForge measurements used a later, separate campaign; the pRESTO observations were retained from the earlier campaign. A monotonic clock measured subprocess launch through successful exit and output completion in fresh directories. Input construction and output validation were outside timing.

After timing, validation required successful exit, the complete expected output SHA-256, and the expected FASTQ record count. ClonoForge completion information and pRESTO pass/fail counts were checked. Speedup is the pRESTO median divided by the ClonoForge median. Warmups are excluded. No benchmark was rerun for this manuscript.

Data and code availability

The public accessions are SRR4026043 and SRR4431771 [2,3]. The official SRR4026043 example is archived on Zenodo [4]. SRR1383459 supplied separate large-group qualification [5]. The evaluated source is identified by the commit above. The earlier v1.0 source archive is a separate snapshot and does not identify this measured binary.

Original code is licensed under MIT OR Apache-2.0. The manuscript package includes measurements, concordance hashes, figure sources, and evidence extracts. Raw reads and repeated consensus outputs are not redistributed here.

ClonoForge 1.0.0 source code is available at https://github.com/Yoonecorn/ClonoForge.

References

1. Vander Heiden JA, et al. pRESTO: a toolkit for processing high-throughput sequencing raw reads of lymphocyte receptor repertoires. Bioinformatics. 2014;30:1930–1932. doi:10.1093/bioinformatics/btu138.

2. NCBI Sequence Read Archive. SRR4026043. BioProject PRJNA338795; experiment SRX2018082.

3. NCBI Sequence Read Archive. SRR4431771. BioProject PRJNA349143; experiment SRX2251665.

4. Immcantation. Official VanderHeiden2017 example workflow and data. Zenodo record 10527842. Archived example.

5. NCBI Sequence Read Archive. SRR1383459. Separate authentic-read group-size qualification.